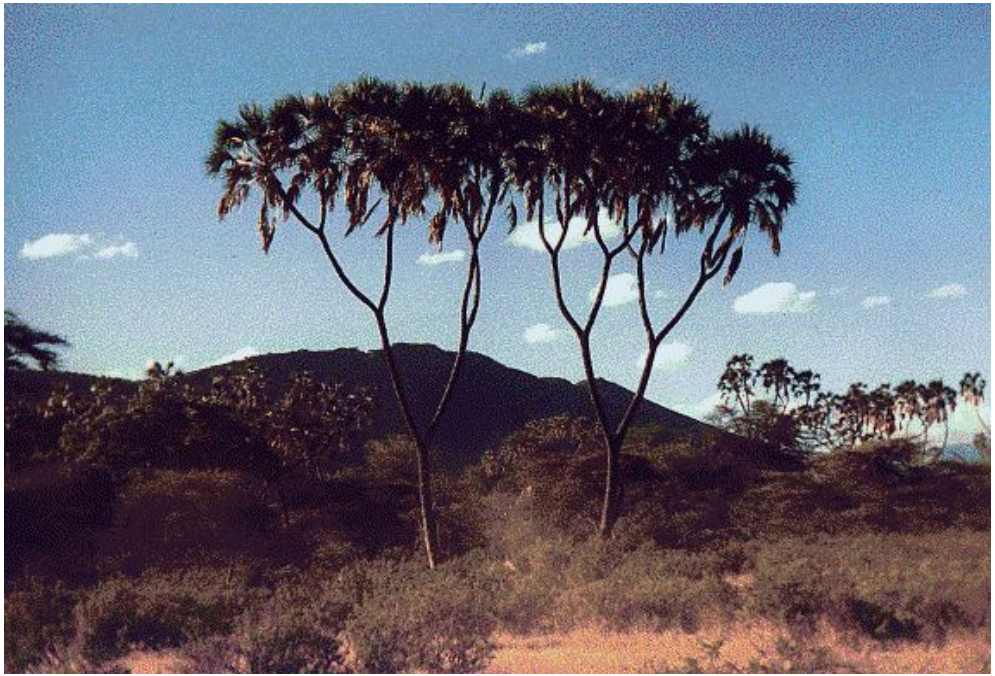


Lecture 16 (Data Structures 2)

Extends, BSTs

CS61B, Fall 2025 @ UC Berkeley

Josh Hug and Peyrin Kao

Quick Reflection on HW07 (based on OH conversation)

Some of you may have noticed IntelliJ suggesting that you write `Comparators` like this:

```
public static Comparator<String> getXComparator() {
    return new Comparator<String>() {
        @Override
        public int compare(String a, String b) {
            int ca = countOf(a, 'x');
            int cb = countOf(b, 'x');
            return ca - cb;
        }
    };
}
```

We have not covered this syntax.

- Creates a class with no name, also known as an “Anonomyous Class”.

Quick Reflection on HW07 (based on OH conversation)

The two code snippets below are identical:

```
public static Comparator<String> getXComparator() {
    return new Comparator<String>() {
        @Override
        public int compare(String a, String b) {
            int ca = countOf(a, 'x');
            int cb = countOf(b, 'x');
            return ca - cb;
        }
    };
}
```

Returns instance of class with no name (i.e. anonymous)

Returns instance of class named XComparator

```
private static class XComparator implements Comparator<String> {
    public int compare(String a, String b) {
        int ca = countOf(a, 'x');
        int cb = countOf(b, 'x');
        return ca - cb;
    }
}

public static Comparator<String> getXComparator() {
    return new XComparator();
}
```

RotatingLL

Lecture 16, CS61B, Fall 2025

Extends

- **RotatingLL**
- TimeSeries

ADTs Review

Binary Search Trees

- Derivation
- Definition
- contains
- Insert
- Hibbard deletion

Sets and Maps (are the same thing)

BST Implementation Tips

Extending a Class

Today we'll cover two topics:

- Extending classes. You'll need this idea for project 4A.
- Implementing a Set (or Map) using a Binary Search Tree.

The Extends Keyword

When a class is a hyponym of an interface, we used **implements.**

- Example: `ArrayList<T> implements List<T>`

instead of an interface

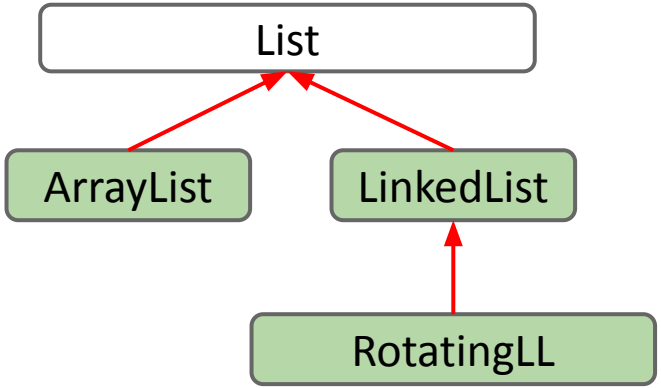
If you want one class to be a hyponym of another *class*, you use **extends.**

We'd like to build RotatingLL that can perform any LinkedList operation as well as:

- rotateLeft(): Moves front item to the back.

Example: Suppose we have [5, 9, 15, 22].

- After rotateLeft: [9, 15, 22, 5]



Demo: Rotating LinkedList

RotatingLL.java

```
public class RotatingLL<Item> {
  public static void main(String[] args) {
    RotatingLL<Integer> rsl = new RotatingLL<>();
    /* Creates List: [10, 11, 12, 13] */
    rsl.addLast(10);
    rsl.addLast(11);
    rsl.addLast(12);
    rsl.addLast(13);

    /* Should be: [11, 12, 13, 10] */
    rsl.rotateLeft();
    System.out.println(rsl.getFirst()); // print 11
  }
}
```

This does not compile. `RotatingLL` is missing the `addLast`, `rotateLeft`, and `getFirst` methods.

Demo: Rotating LinkedList

RotatingLL.java

```
public class RotatingLL<Item> extends LinkedList<Item> {
  public static void main(String[] args) {
    RotatingLL<Integer> rsl = new RotatingLL<>();
    /* Creates List: [10, 11, 12, 13] */
    rsl.addLast(10);
    rsl.addLast(11);
    rsl.addLast(12);
    rsl.addLast(13);

    /* Should be: [11, 12, 13, 10] */
    rsl.rotateLeft();
    System.out.println(rsl.getFirst()); // print 11
  }
}
```

Now the compiler knows that a `RotatingLL` is a `LinkedList`, so `RotatingLL` inherits the `addLast` and `getFirst` methods from the `LinkedList` class.

The `rotateLeft` method is still missing.

Demo: Rotating SLList

RotatingLL.java

```
public class RotatingLL<Item> extends LinkedList<Item> {
  public static void main(String[] args) {
    RotatingLL<Integer> rsl = new RotatingLL<>();
    rsl.addLast(10); rsl.addLast(11); rsl.addLast(12); rsl.addLast(13);

    /* Rotates from [10, 11, 12, 13] to [13, 10, 11, 12] */
    rsl.rotateLeft();
    System.out.println(rsl.getFirst()); // print 11
   }

   /** Rotates list to the left. */
   public void rotateLeft() {


   }
}
```

Demo: Rotating SLList

RotatingLL.java

```
public class RotatingLL<Item> extends LinkedList<Item> {
  public static void main(String[] args) {
    RotatingLL<Integer> rsl = new RotatingLL<>();
    rsl.addLast(10); rsl.addLast(11); rsl.addLast(12); rsl.addLast(13);

    /* Rotates from [10, 11, 12, 13] to [13, 10, 11, 12] */
    rsl.rotateLeft();
    rsl.print();
   }

   /** Rotates list to the left. */
   public void rotateLeft() {
       Item oldFirst = removeFirst();

   }
}
```

Demo: Rotating SLList

RotatingLL.java

```
public class RotatingLL<Item> extends LinkedList<Item> {
  public static void main(String[] args) {
    RotatingLL<Integer> rsl = new RotatingLL<>();
    rsl.addLast(10); rsl.addLast(11); rsl.addLast(12); rsl.addLast(13);

    /* Rotates from [10, 11, 12, 13] to [13, 10, 11, 12] */
    rsl.rotateLeft();
    rsl.print();
   }

   /** Rotates list to the left. */
   public void rotateLeft() {
       Item oldFirst = removeFirst();
       addLast(oldFirst);
   }
}
```

RotatingLL

```
public class RotatingLL<Item> extends LinkedList<Item> {
    public void rotateLeft() {
        Item oldFirst = removeFirst();
        addLast(oldFirst);
    }
}
```

Because of **extends**, `RotatingLL` inherits all members of `LinkedList`:

- All instance and static variables.
- All methods.
- All nested classes.

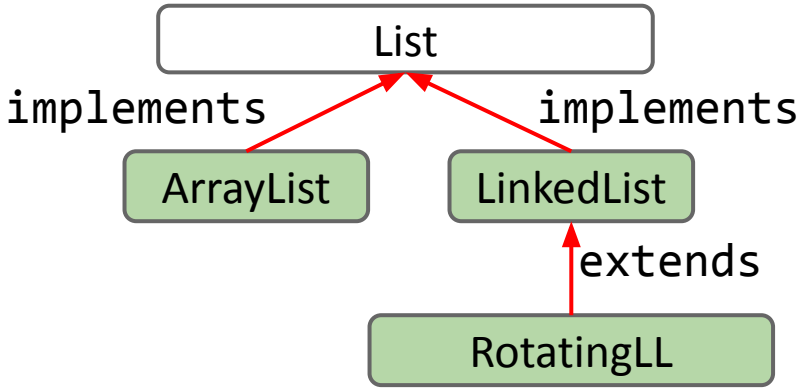
… but members may be private and thus inaccessible!

Note: The constructor for superclass (`LinkedList`) is implicitly called when you use the subclass constructor (`new RotatingLL()`).

Extending Classes Summary

We use **`extends`** to say that `RotatingLL` is a **subclass** of `LinkedList`.

- `RotatingLL` inherits all members of `LinkedList`.
- Can override `LinkedList` methods.
- Can invoke `LinkedList` methods, variables, or constructors using **`super`**.
  - We won't really use this in 61B. May see on discussion worksheets.
- If you do not manually invoke `SLList`'s constructor using e.g. `super()`, it will be called for your implicitly.
  - This ensures that, e.g. any required `sentinel` variable is defined.



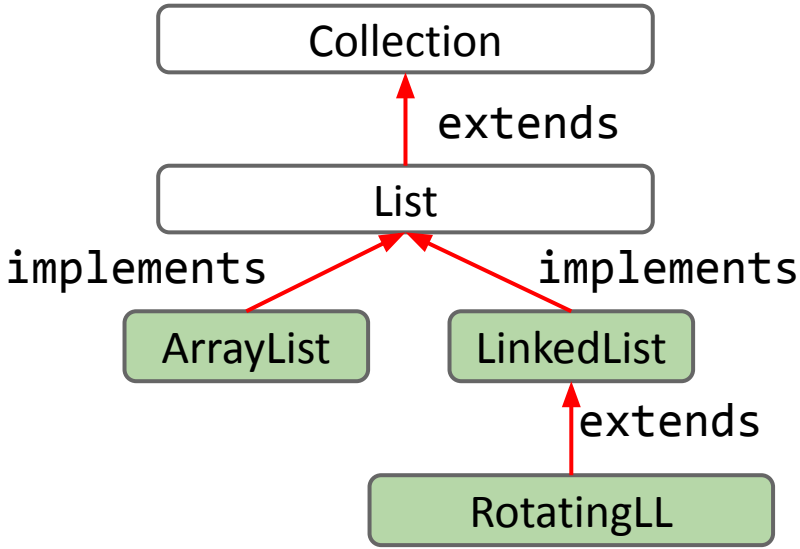
Extends vs. Implements.

To me (Josh), `extends` and `implements` are nearly exact synonyms.

- Both establish hypernym / hyponym relationships. Subtype inherits all members of the supertype.
- `Implement` is used between classes / interfaces.
- `Extends` is used between classes / classes and interfaces / interfaces.

Why not just use extends for both?

- `Implements` is something special!
- `Implements`: Crossing the boundary from abstract to concrete.



TimeSeries

Lecture 16, CS61B, Fall 2025

Extends

- RotatingLL
- **TimeSeries**

Binary Search Trees

- Today's goal: Implement the Set (and Map) ADT
- Derivation
- Definition
- contains
- Insert
- Hibbard deletion

Sets and Maps (are the same thing)

BST Implementation Tips

Project 4A

On project 4A, you'll build a class called `TimeSeries`.

- Tracks values of some quantity over time.
- Desired behavior below.
  - Example: GDP of the US in 1990 is $5.963 trillion.

```
public static void main(String[] args) {
   TimeSeries usGDP = new TimeSeries();
   usGDP.put(1990, 5.963);
   usGDP.put(1991, 6.158);
   usGDP.put(1992, 6.520);
   usGDP.put(1993, 6.858);
   usGDP.put(1994, 7.287);
   usGDP.put(1995, 7.639);
   usGDP.put(1996, 8.073);
   usGDP.put(1997, 8.577);
   System.out.println(usGDP.get(1990));
}
```

How do we achieve this?

- Could design our own `TimeSeries` data structure from scratch and implement `put` and `get`.

```
public static void main(String[] args) {
   TimeSeries usGDP = new TimeSeries();
   usGDP.put(1990, 5.963);
   usGDP.put(1991, 6.158);
   usGDP.put(1992, 6.520);
   usGDP.put(1993, 6.858);
   usGDP.put(1994, 7.287);
   usGDP.put(1995, 7.639);
   usGDP.put(1996, 8.073);
   usGDP.put(1997, 8.577);
   System.out.println(usGDP.get(1990));
}
```

How do we achieve this?

- Could design our own `TimeSeries` data structure from scratch and implement `put` and `get`.
- How can we avoid all that work?

```
public static void main(String[] args) {
   TimeSeries usGDP = new TimeSeries();
   usGDP.put(1990, 5.963);
   usGDP.put(1991, 6.158);
   usGDP.put(1992, 6.520);
   usGDP.put(1993, 6.858);
   usGDP.put(1994, 7.287);
   usGDP.put(1995, 7.639);
   usGDP.put(1996, 8.073);
   usGDP.put(1997, 8.577);
   System.out.println(usGDP.get(1990));
}
```

How do we achieve this?

- Could design our own `TimeSeries` data structure from scratch and implement `put` and `get`.
- How can we avoid all that work? Extend `TreeMap` (or `HashMap`). Let's try.

```
public static void main(String[] args) {
   TimeSeries usGDP = new TimeSeries();
   usGDP.put(1990, 5.963);
   usGDP.put(1991, 6.158);
   usGDP.put(1992, 6.520);
   usGDP.put(1993, 6.858);
   usGDP.put(1994, 7.287);
   usGDP.put(1995, 7.639);
   usGDP.put(1996, 8.073);
   usGDP.put(1997, 8.577);
   System.out.println(usGDP.get(1990));
}
```

TimeSeries Implementation

TimeSeries.java

```
public class TimeSeries {
    public static void main(String[] args) {
        TimeSeries usGDP = new TimeSeries();
        usGDP.put(1990, 5.963);
        usGDP.put(1991, 6.158);
        usGDP.put(1992, 6.520);
        usGDP.put(1993, 6.858);
        usGDP.put(1994, 7.287);
        usGDP.put(1995, 7.639);
        usGDP.put(1996, 8.073);
        usGDP.put(1997, 8.577);
        System.out.println(usGDP.get(1990));
    }
}
```

TimeSeries Implementation

TimeSeries.java

```
public class TimeSeries extends TreeMap<Integer, Double> {
    public static void main(String[] args) {
        TimeSeries usGDP = new TimeSeries();
        usGDP.put(1990, 5.963);
        usGDP.put(1991, 6.158);
        usGDP.put(1992, 6.520);
        usGDP.put(1993, 6.858);
        usGDP.put(1994, 7.287);
        usGDP.put(1995, 7.639);
        usGDP.put(1996, 8.073);
        usGDP.put(1997, 8.577);
        System.out.println(usGDP.get(1990));
    }
}
```

Your Job on Project 4A

On project 4A, you'll add more `TimeSeries` methods such as `plus`.

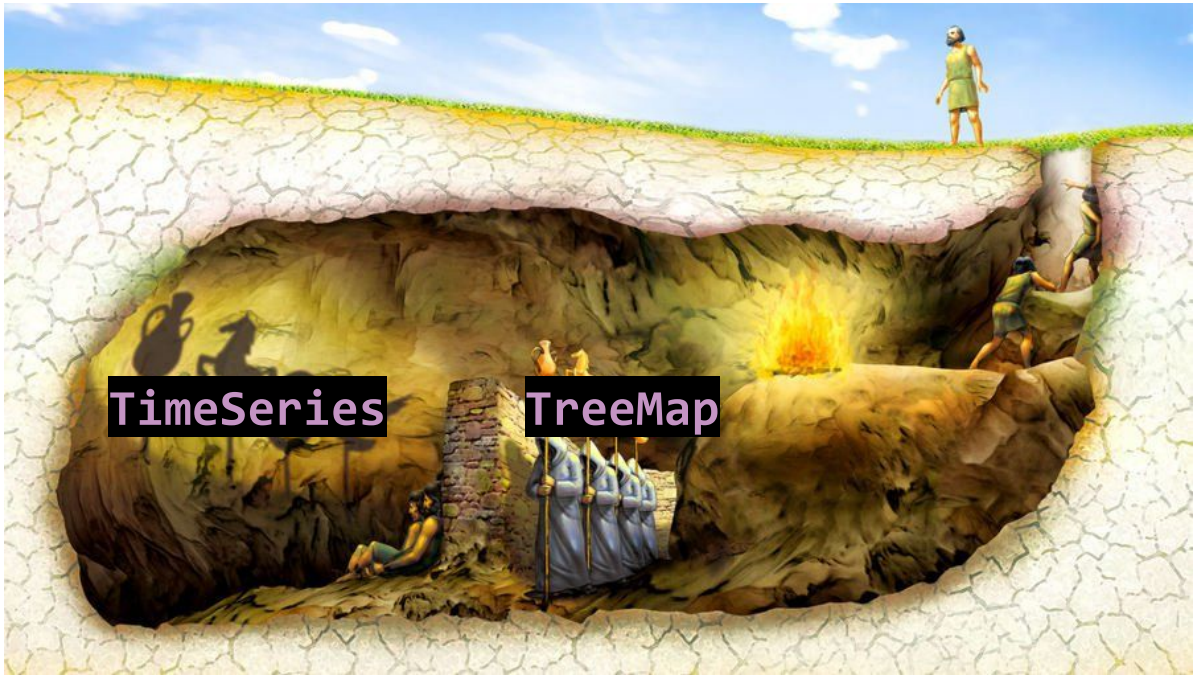
- The nice thing is that you don't have to know anything at all about how `put` and `get` work! You can just trust them.

```
TimeSeries usGDP = new TimeSeries();
TimeSeries chinaGDP = new TimeSeries();
...
System.out.println(usGDP.plus(chinaGDP));
```

Layers of Abstraction

It's the usual story.

- We write the `TimeSeries` class, relying on Josh Bloch and Doug Lea to implement the underlying TreeMap operations (`get` and `put`).



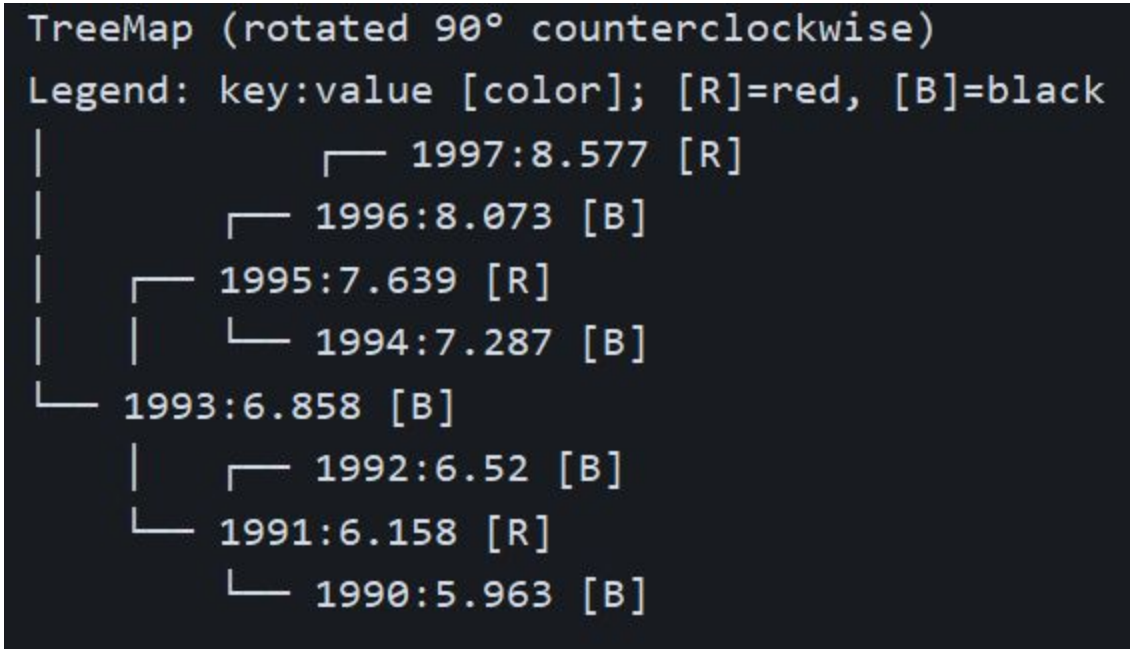
Yeah, but What's in the TreeMap?

Technically, our `TimeSeries` class inherits all members of the TreeMap.

- But these members are private!
- Technically everything is there in some sort of tree starting at `root`.
  - IntellIJ yells at us if we try to access `root`.

The Reflections Library

Note: There is a library in Java that lets us ignore access modifiers. Using this library, we can actually get access to the underlying tree, which looks like this:



Over the next three lectures, we'll work to understand how this TreeMap works.

Today's goal: Implement the Set (and Map) ADT

Lecture 16, CS61B, Fall 2025

Extends

- RotatingLL
- TimeSeries

Binary Search Trees

- **Today's goal: Implement the Set (and Map) ADT**
- Derivation
- Definition
- contains
- Insert
- Hibbard deletion

Sets and Maps (are the same thing)

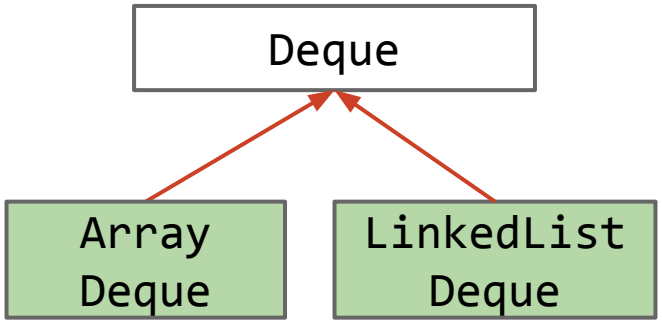
BST Implementation Tips

Abstract Data Types

An **Abstract Data Type (ADT)** is defined only by its operations, not by its implementation.

Deque ADT:

- `addFirst(Item x);`
- `addLast(Item x);`
- `boolean isEmpty();`
- `int size();`
- `printDeque();`
- `Item removeFirst();`
- `Item removeLast();`
- `Item get(int index);`

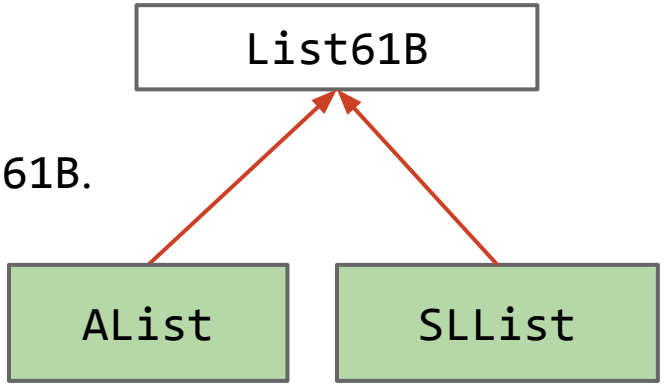


ArrayDeque and LinkedList Deque are implementations of the Deque ADT.

Interfaces vs. Implementation

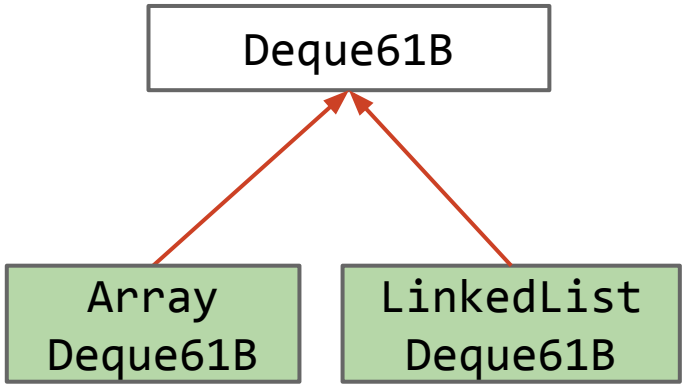
In class:

- Developed `ALists` and `SLLists`.
- Created an interface `List61B`.
  - Modified `AList` and `SLList` to implement `List61B`.
  - `List61B` provided default methods.



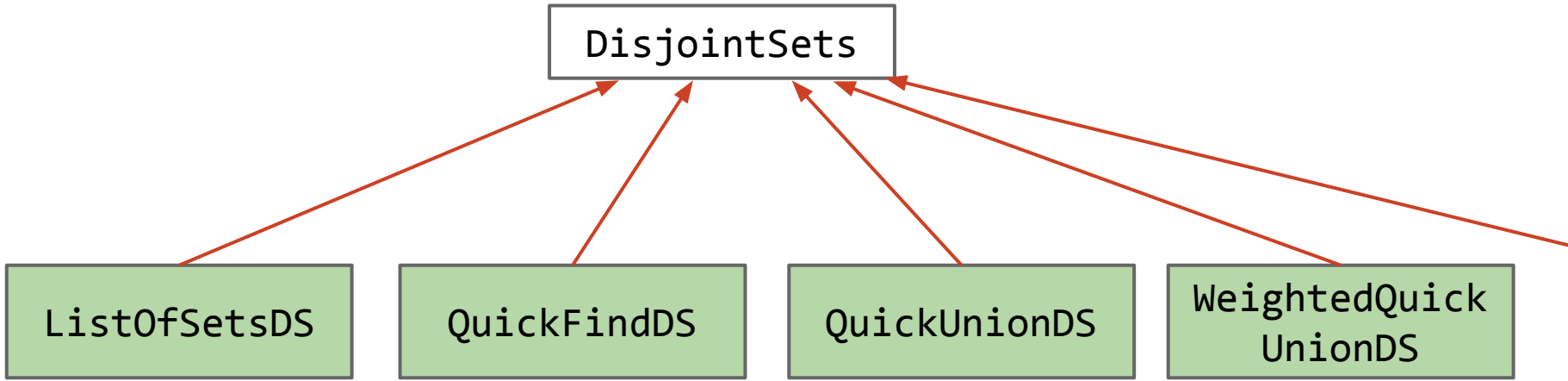
In projects:

- Developed `ArrayDeque` and `LinkedListDeque`.
  - Each class implemented the `Deque` interface.



Interfaces vs. Implementation

With the `DisjointSets` ADT, we saw a richer set of possible implementations.

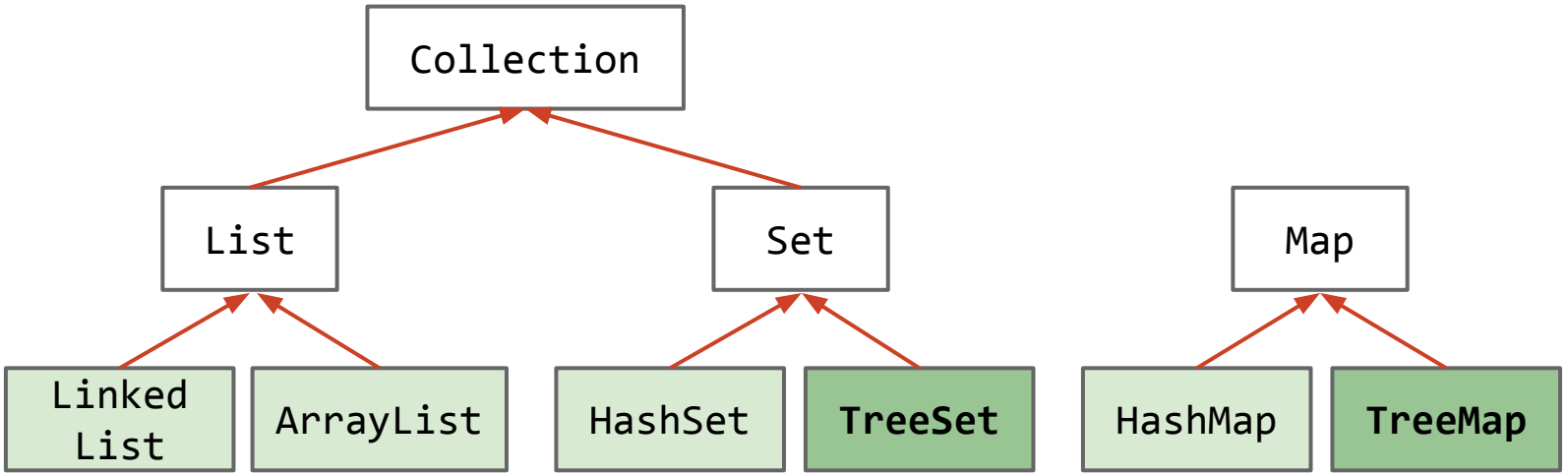


Java Libraries

The built-in java.util package provides a number of useful:

- Interfaces: ADTs (lists, sets, maps, priority queues, etc.) and other stuff.
- Implementations: Concrete classes you can use.

In this lecture, we'll learn the basic ideas behind the `TreeSet` and `TreeMap`.



Binary Search Trees: Derivation

Lecture 16, CS61B, Fall 2025

Extends

- RotatingLL
- TimeSeries

Binary Search Trees

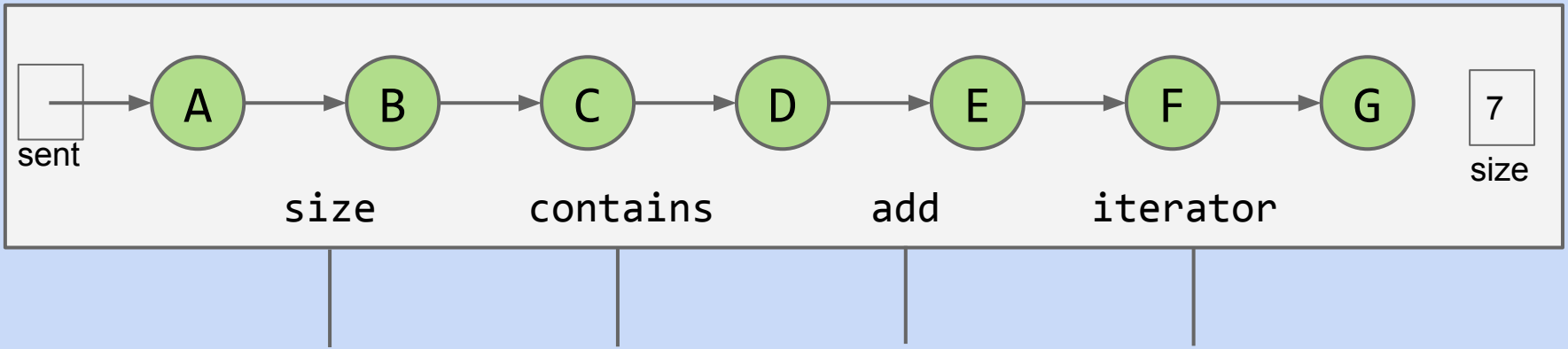
- Today's goal: Implement the Set (and Map) ADT
- **Derivation**
- Definition
- contains
- Insert
- Hibbard deletion

Sets and Maps (are the same thing)

BST Implementation Tips

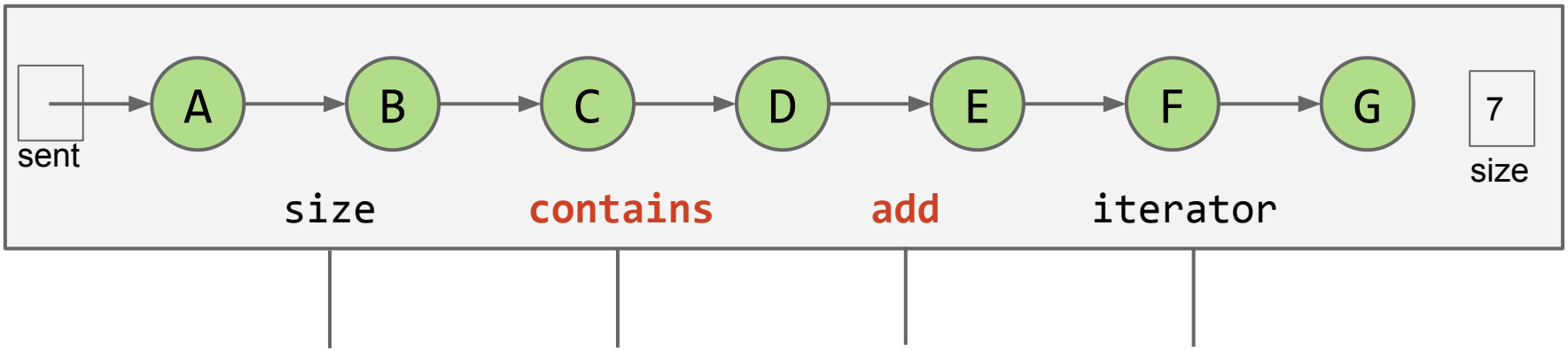
Analysis of an OrderedLinkedListSet<Character>

In an earlier lecture, we implemented a set based on unordered arrays. For the ***order linked list*** set implementation below, name an operation that takes worst case linear time, i.e. Θ(N).



Analysis of an OrderedLinkedListSet<Character>

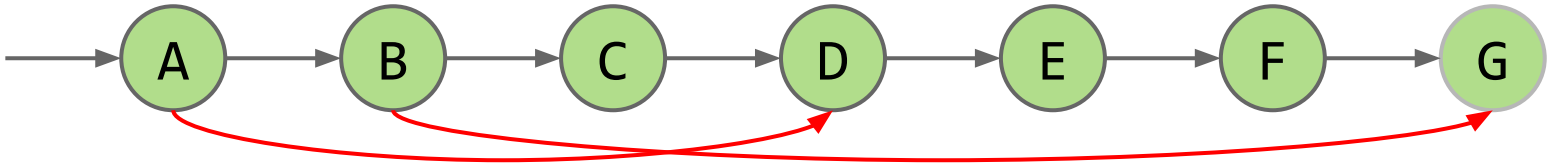
In an earlier lecture, we implemented a set based on unordered arrays. For the ***order linked list*** set implementation below, name an operation that takes worst case linear time, i.e. Θ(N).



Optimization Idea #1: Extra Links

Fundamental Problem: Slow search, even though it's in order.

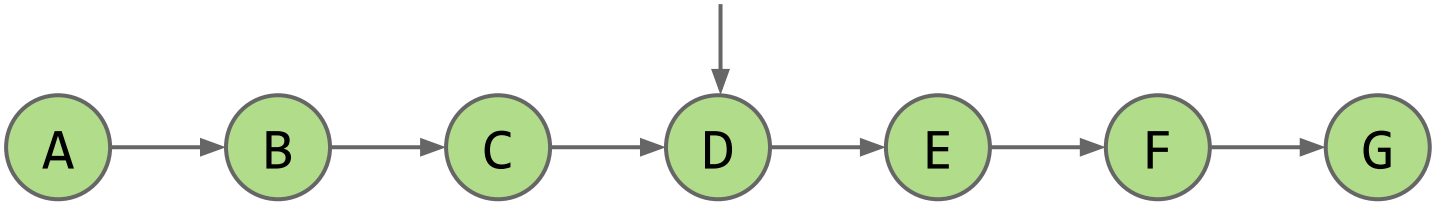
- Add (random) express lanes. Skip List (won't discuss in 61B)



Optimization Idea #2: Change the Entry Point

Fundamental Problem: Slow search, even though it's in order.

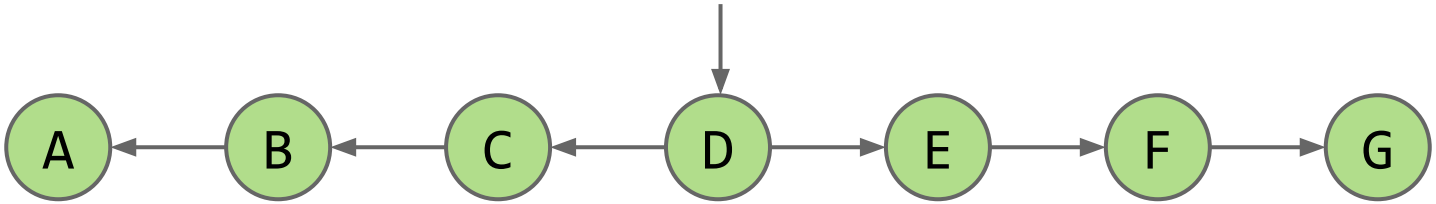
- Move pointer to middle.



Optimization Idea #2: Change the Entry Point, Flip Links

Fundamental Problem: Slow search, even though it's in order.

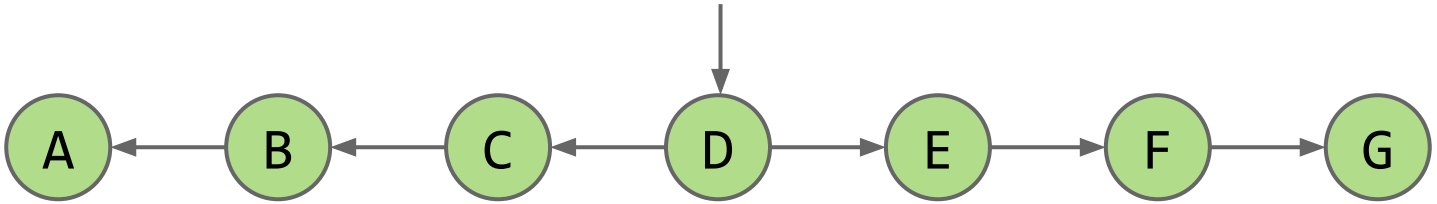
- Move pointer to middle and flip left links. Halved search time!



Optimization Idea #2: Change the Entry Point, Flip Links

Fundamental Problem: Slow search, even though it's in order.

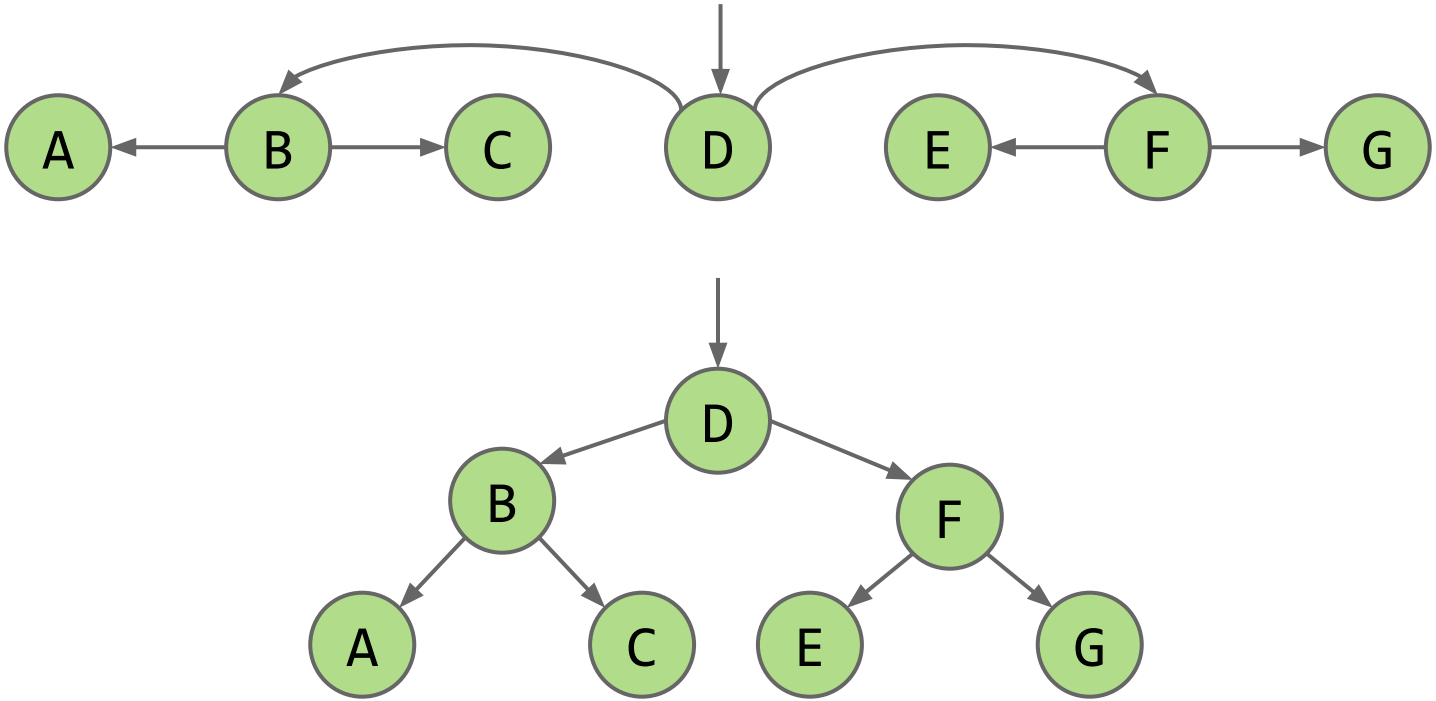
- How do we do even better?
- Dream big!



Optimization Idea #2: Change Entry Point, Flip Links, Allow Big Jumps

Fundamental Problem: Slow search, even though it's in order.

- How do we do better?



Binary Search Trees: Definition

Lecture 16, CS61B, Fall 2025

Extends

- RotatingLL
- TimeSeries

Binary Search Trees

- Today's goal: Implement the Set (and Map) ADT
- Derivation
- **Definition**
- contains
- Insert
- Hibbard deletion

Sets and Maps (are the same thing)

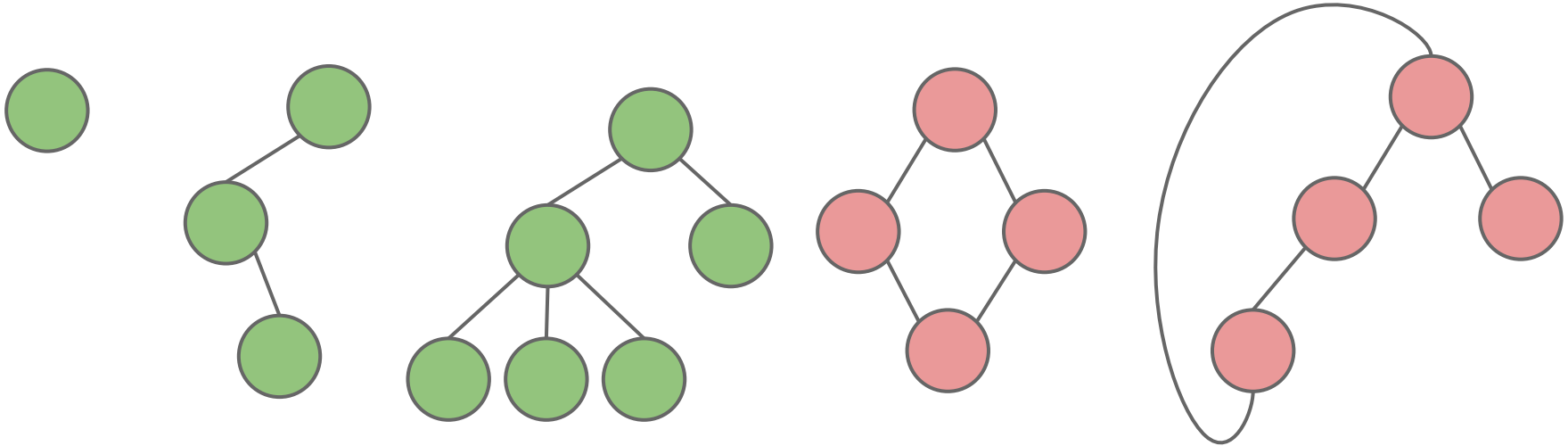
BST Implementation Tips

Tree

A tree consists of:

- A set of nodes.
- A set of edges that connect those nodes.
  - Constraint: There is exactly one path between any two nodes.

Green structures below are trees. Pink ones are not.



Rooted Trees and Rooted Binary Trees

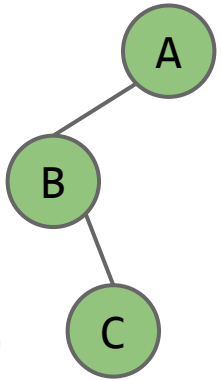
In a rooted tree, we call one node the root.

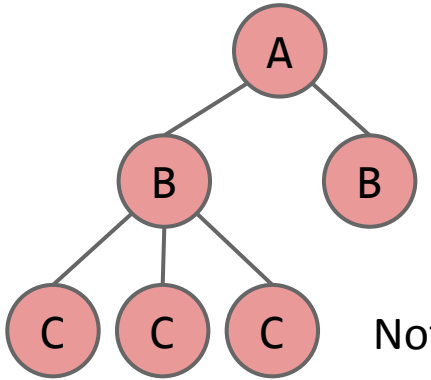
- Every node N except the root has exactly one parent, defined as the first node on the path from N to the root.
- Unlike (most) real trees, the root is usually depicted at the top of the tree.
- A node with no child is called a leaf.

In a rooted binary tree, every node has either 0, 1, or 2 children (subtrees).

For each of these:

- A is the root.
- B is a child of A. (and C of B)
- A is a parent of B. (and B of C)





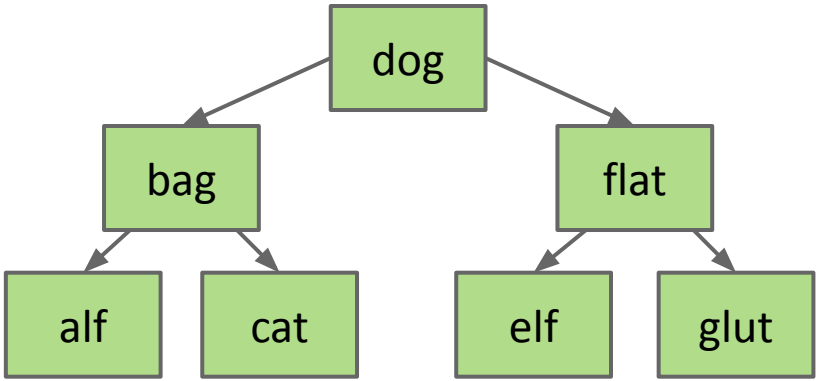
Not binary!

Binary Search Trees

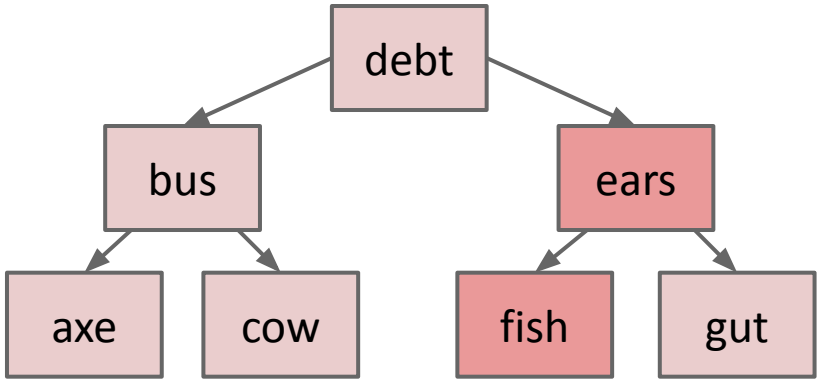
A binary search tree is a rooted binary tree with the BST property.

BST Property. For every node X in the tree:

- Every key in the **left** subtree is **less** than X's key.
- Every key in the **right** subtree is **greater** than X's key.



Binary Search Tree



Binary Tree, but not a Binary Search Tree

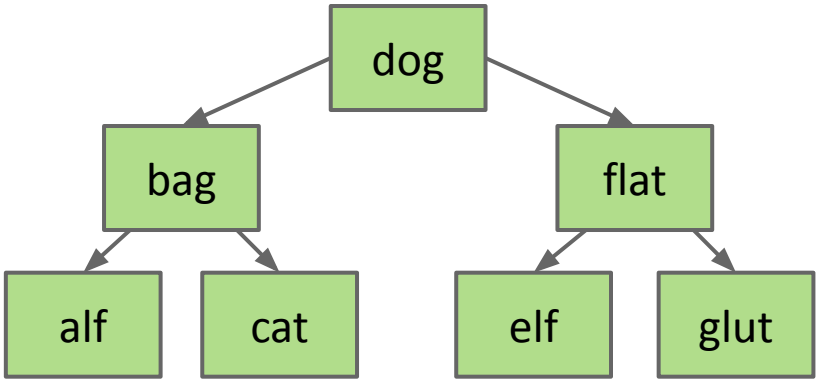
Binary Search Trees

Ordering must be complete, transitive, and antisymmetric. Given keys p and q:

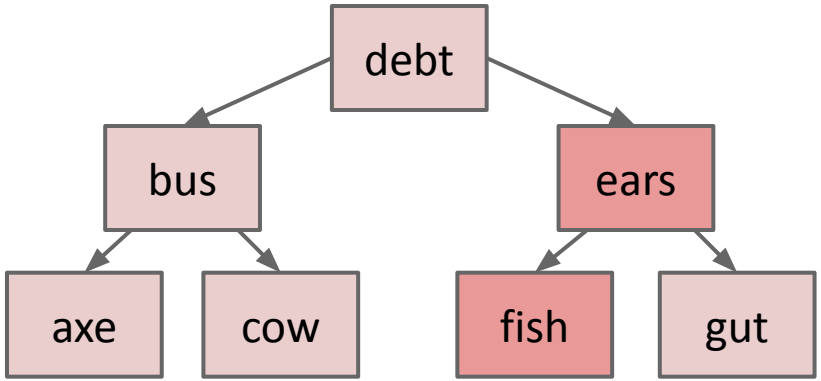
- Exactly one of $p < q$ and $q < p$ are true.
- $p < q$ and $q < r$ imply $p < r$.

One consequence of these rules: No duplicate keys allowed!

- Keeps things simple. Most real world implementations follow this rule.



Binary Search Tree



Binary Tree, but not a Binary Search Tree

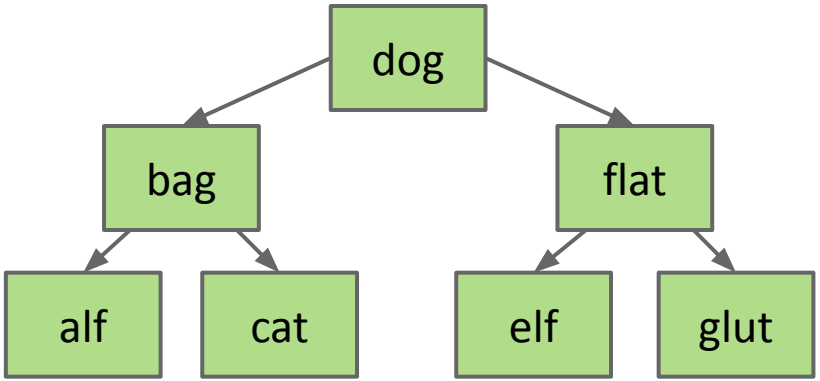
Binary Search Trees

Ordering must be complete, transitive, and antisymmetric. Given keys p and q:

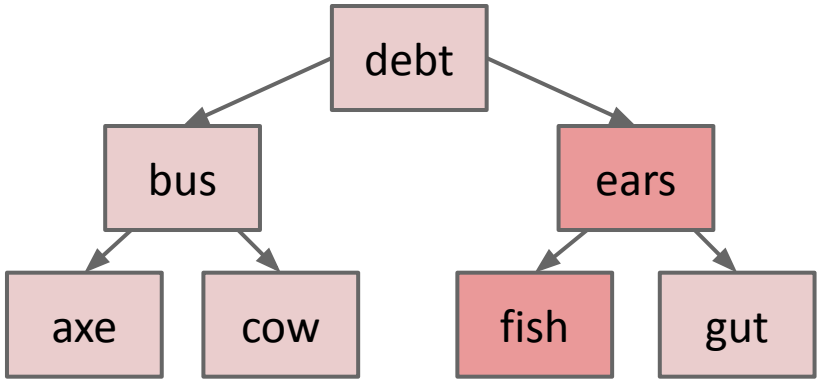
- Exactly one of $p \prec q$ and $q \prec p$ are true.
- $p \prec q$ and $q \prec r$ imply $p \prec r$.

$\prec$ is used in place of < to remind ourselves that ordering is arbitrary.

- For example, could have used string length to define $\prec$ instead.



Binary Search Tree



Binary Tree, but not a Binary Search Tree

contains

Lecture 16, CS61B, Fall 2025

Extends

- RotatingLL
- TimeSeries

Binary Search Trees

- Today's goal: Implement the Set (and Map) ADT
- Derivation
- Definition
- **contains**
- Insert
- Hibbard deletion

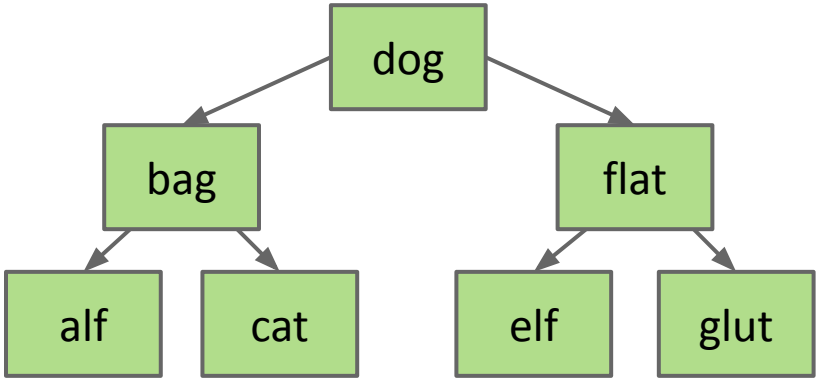
Sets and Maps (are the same thing)

BST Implementation Tips

Finding a searchKey in a BST (come back to this for the BST lab)

If searchKey equals T.key, return.

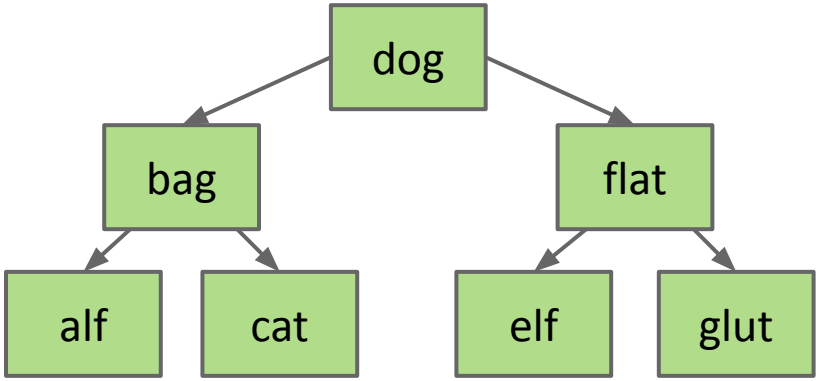
- If searchKey < T.key, search T.left.
- If searchKey > T.key, search T.right.



Finding a searchKey in a BST

If searchKey equals T.key, return.

- If searchKey < T.key, search T.left.
- If searchKey > T.key, search T.right.



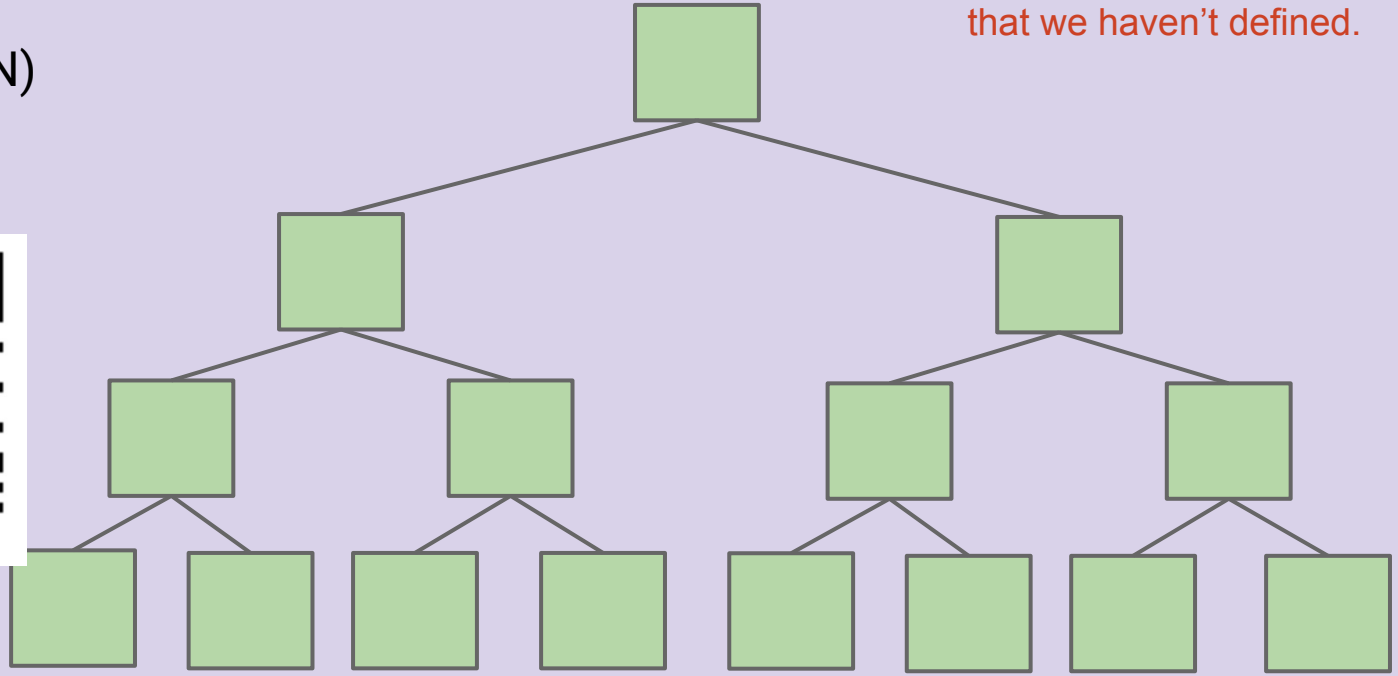

```
static BST find(BST T, Key sk) {
   if (T == null)
       return null;
   if (sk.equals(T.key))
       return T;
   else if (sk < T.key)
       return find(T.left, sk);
  else
       return find(T.right, sk);
}
```

BST Search: http://yellkey.com/maybe

What is the runtime to complete a search on a “bushy” BST in the worst case, where N is the number of nodes?

A. $\Theta(\log N)$
B. $\Theta(N)$
C. $\Theta(N \log N)$
D. $\Theta(N^2)$
E. $\Theta(2^N)$

“bushiness” is an intuitive concept that we haven’t defined.

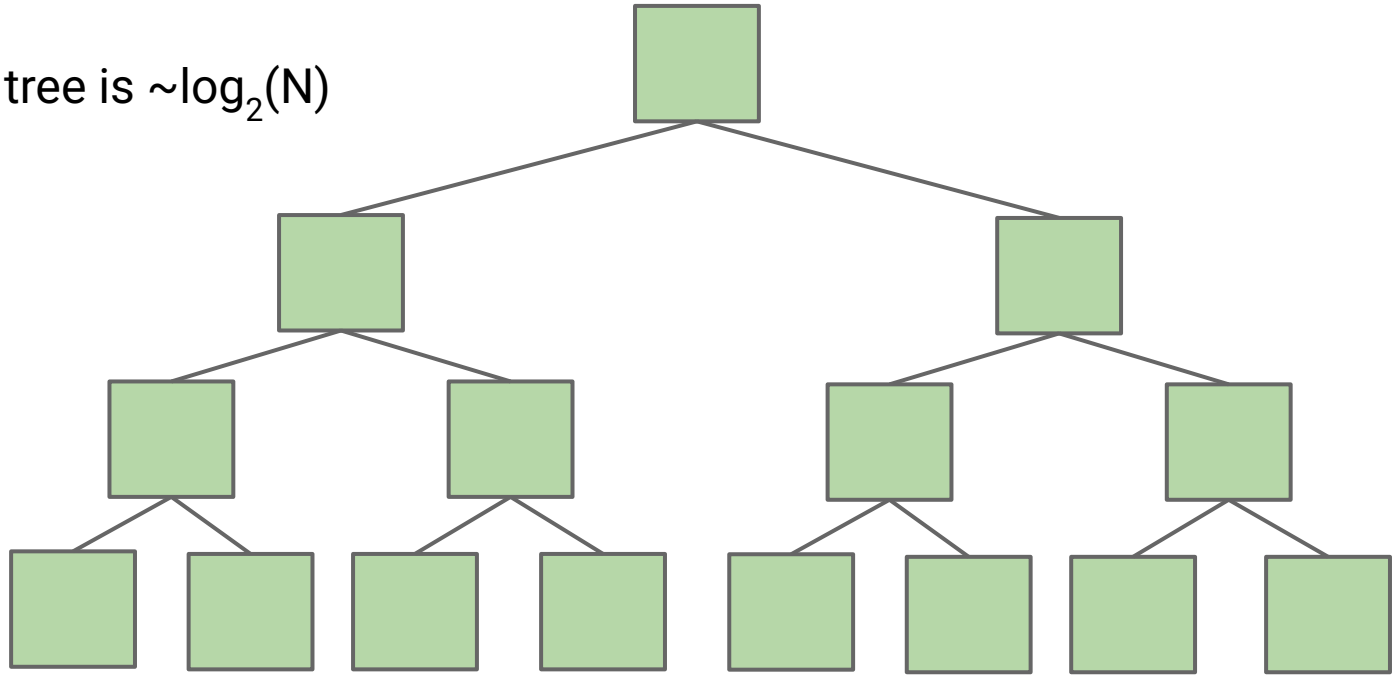


BST Search

What is the runtime to complete a search on a “bushy” BST in the worst case, where N is the number of nodes.

A. Θ(log N)

Height of the tree is ~$\log_2(N)$



BSTs

Bushy BSTs are extremely fast.

- At 1 microsecond per operation, can find something from a tree of size 10^{300000} in one second.

Much (perhaps most?) computation is dedicated towards finding things in response to queries.

- It's a good thing that we can do such queries almost for free.

insert

Lecture 16, CS61B, Fall 2025

Extends

- RotatingLL
- TimeSeries

Binary Search Trees

- Today's goal: Implement the Set (and Map) ADT
- Derivation
- Definition
- contains
- **Insert**
- Hibbard deletion

Sets and Maps (are the same thing)

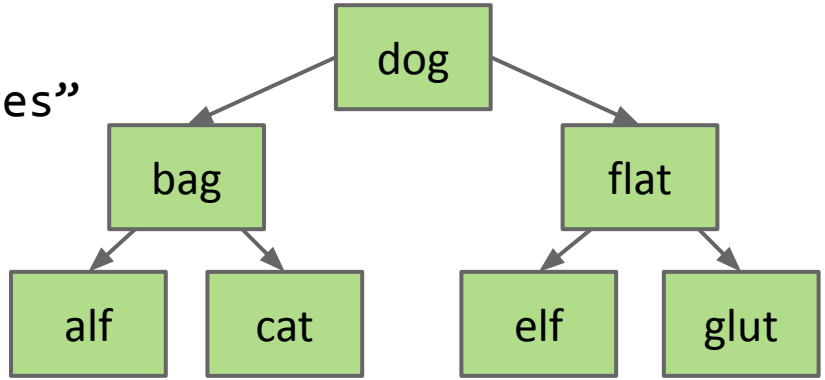
BST Implementation Tips

Inserting a New Key into a BST

Search for key.

- If found, do nothing.
- If not found:
  - Create new node.
  - Set appropriate link.

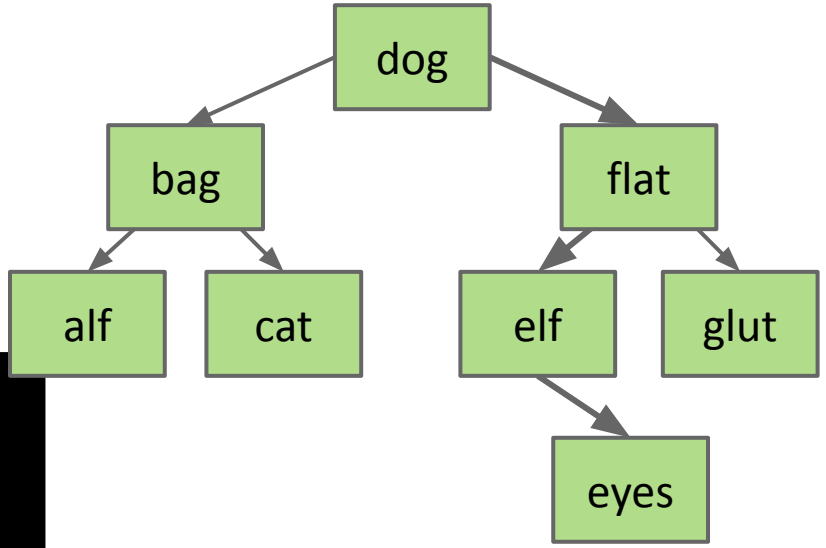
Example:
`insert “eyes”`



Inserting a New Key into a BST

Search for key.

- If found, do nothing.
- If not found:
  - Create new node.
  - Set appropriate link.




```
static BST insert(BST T, Key ik) {
   if (T == null)
       return new BST(ik);
   if (ik < T.key)
     T.left = insert(T.left, ik);
 else if (ik > T.key)
     T.right = insert(T.right, ik);
   return T;
}
```

Arms length recursion: A common rookie bad habit to avoid:

```
if (T.left == null)
  T.left = new BST(ik);
else if (T.right == null)
  T.right = new BST(ik);
```

Avoid Arms-Length Recursion

```
if (T.left.left == null)
  T.left.left = new BST(ik);
else if (T.left.right == null)
  T.left.right = new BST(ik);
else if (T.right.left == null)
  T.right.left = new BST(ik);
else if (T.right.right == null)
  T.right.right = new BST(ik);
```

This base case is too complicated.
The recursion can take us further.

```
if (T.left == null)
  T.left = new BST(ik);
else if (T.right == null)
  T.right = new BST(ik);
```

Better, but still not the best base case.
Avoid arms-length recursion!

```
if (T == null)
 return new BST(ik);
```

The best base case.

Hibbard deletion

Lecture 16, CS61B, Fall 2025

Extends

- RotatingLL
- TimeSeries

Binary Search Trees

- Today's goal: Implement the Set (and Map) ADT
- Derivation
- Definition
- contains
- Insert
- **Hibbard deletion**

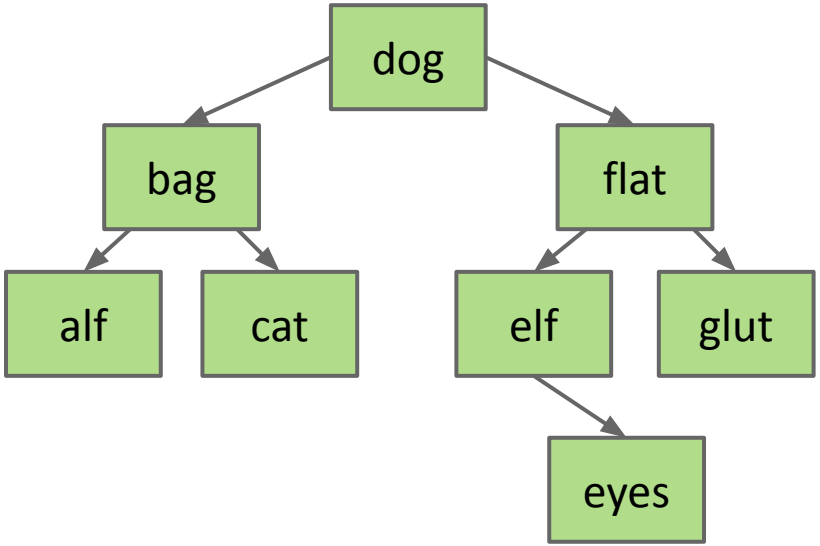
Sets and Maps (are the same thing)

BST Implementation Tips

Deleting from a BST

3 Cases:

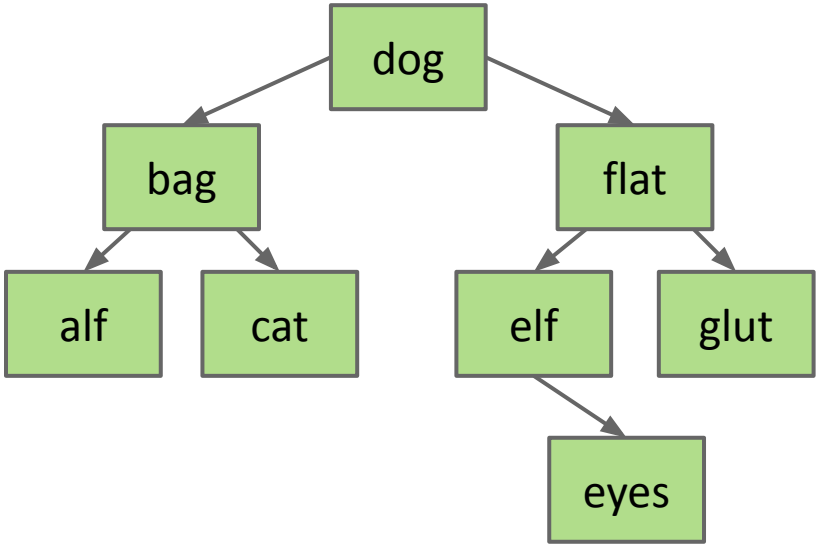
- Deletion key has no children.
- Deletion key has one child.
- Deletion key has two children.



Case 1: Deleting from a BST: Key with no Children

Deletion key has no children (“glut”):

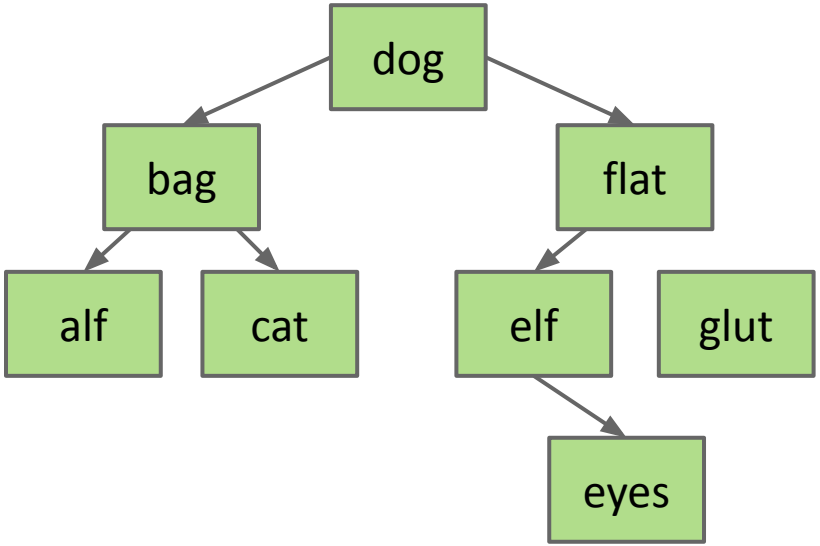
- Just sever the parent’s link.
- What happens to “glut” node?



Case 1: Deleting from a BST: Key with no Children

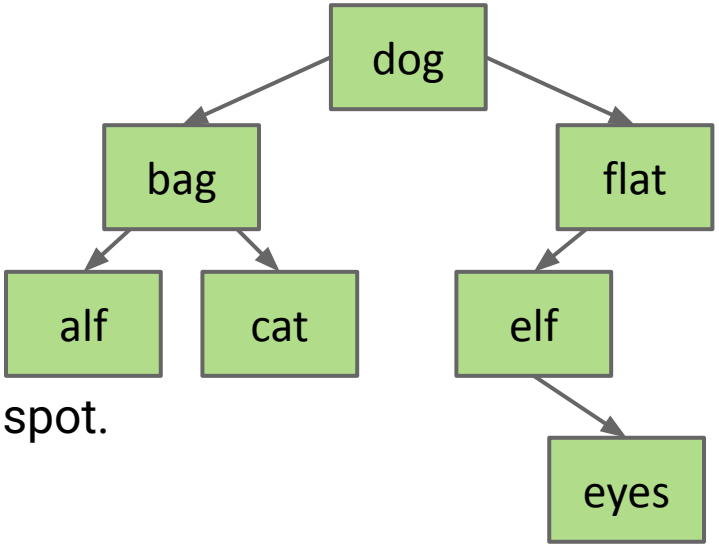
Deletion key has no children (“glut”):

- Just sever the parent’s link.
- What happens to “glut” node?
  - Garbage collected.



Case 2: Deleting from a BST: Key with one Child

Example: delete("flat"):



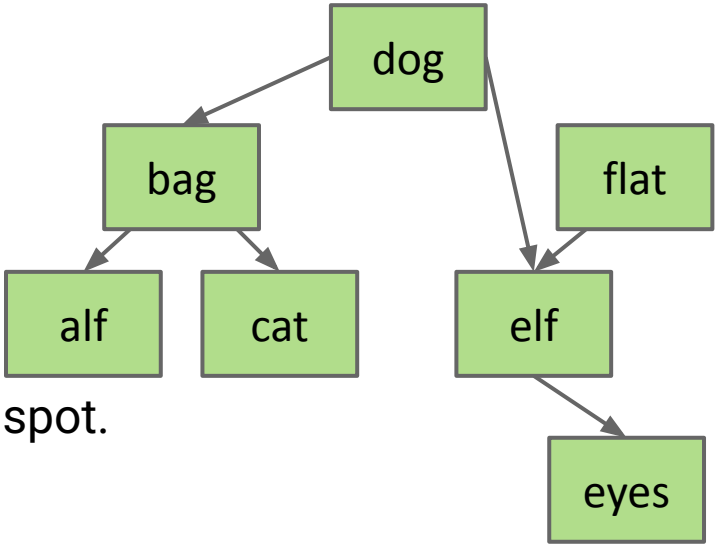
Goal:

- Maintain BST property.
- Flat's child definitely larger than dog.
  - Safe to just move that child into flat's spot.

Thus: Move flat's parent's pointer to flat's child.

Case 2: Deleting from a BST: Key with one Child

Example: delete("flat"):



Goal:

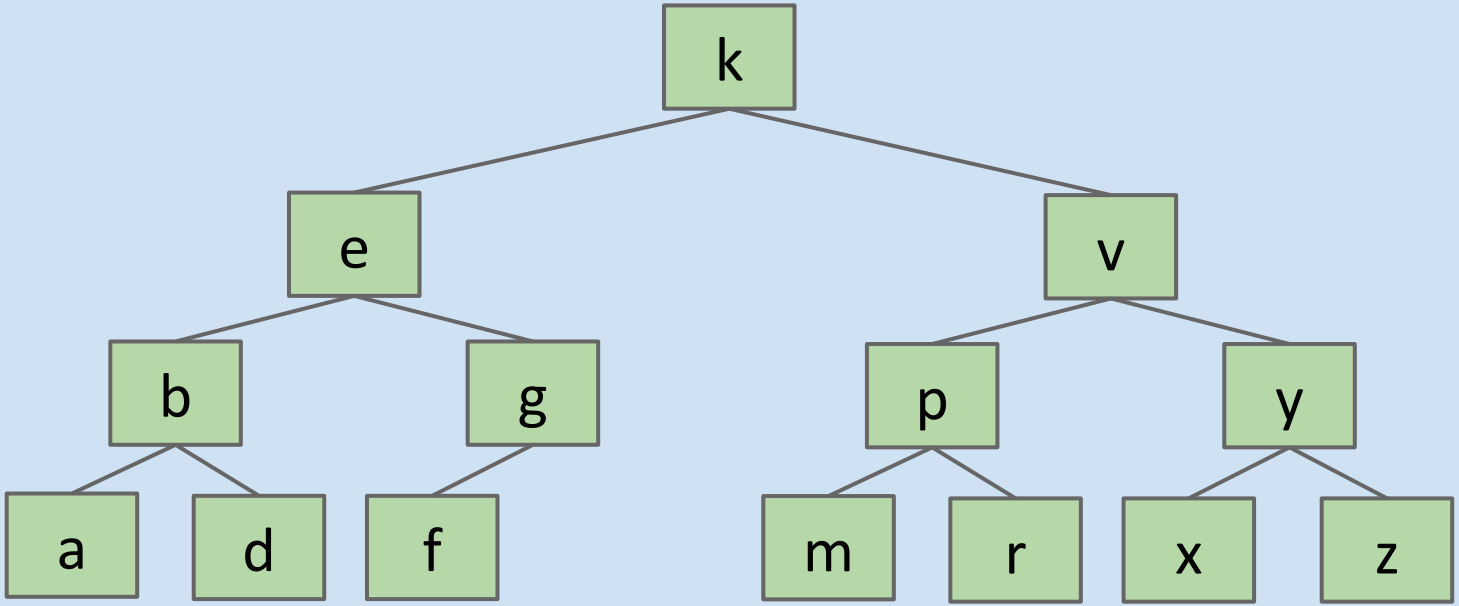
- Maintain BST property.
- Flat's child definitely larger than dog.
  - Safe to just move that child into flat's spot.

Thus: Move flat's parent's pointer to flat's child.

- Flat will be garbage collected (along with its instance variables).

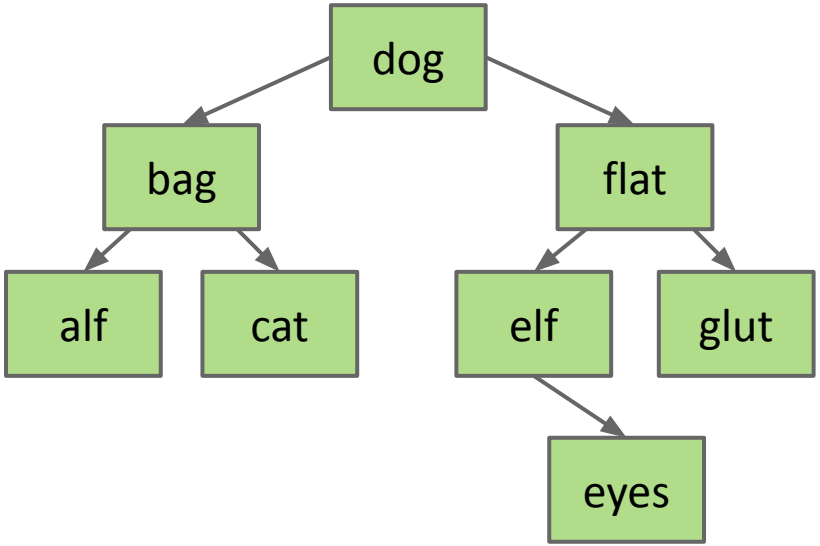
Hard Challenge

Delete k.



Case 3: Deleting from a BST: Deletion with two Children (Hibbard)

Example: delete(“dog”)



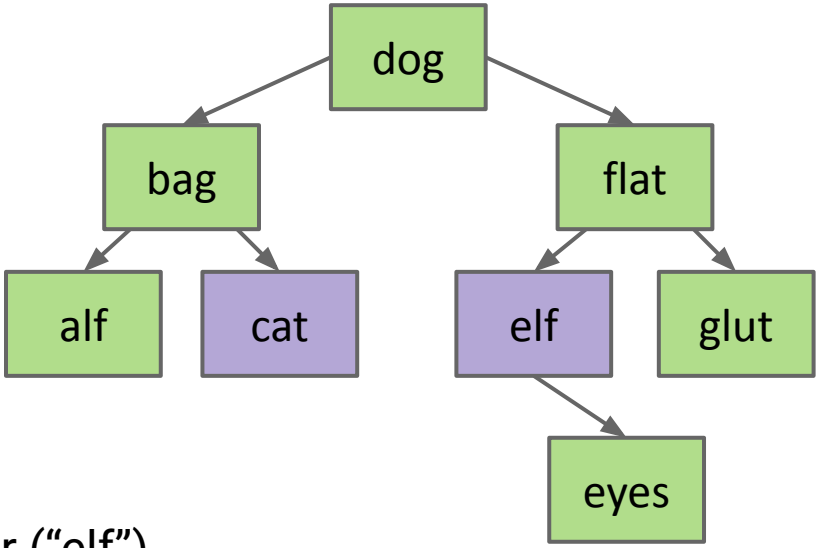
Goal:

- Find a new root node.
- Must be > than everything in left subtree.
- Must be < than everything right subtree.

Would bag work?

Case 3: Deleting from a BST: Deletion with two Children (Hibbard)

Example: delete("dog")



Goal:

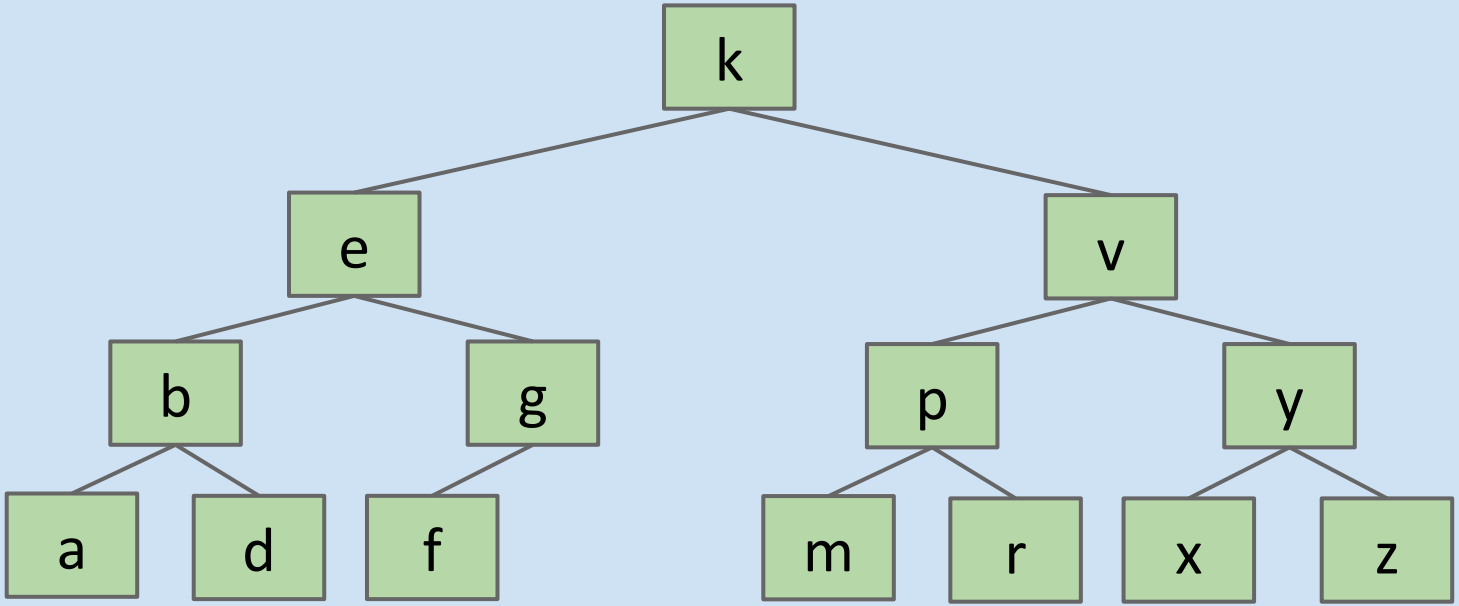
- Find a new root node.
- Must be > than everything in left subtree.
- Must be < than everything right subtree.

Choose either predecessor ("cat") or successor ("elf").

- Delete "cat" or "elf", and stick new copy in the root position:
  - This deletion guaranteed to be either case 1 or 2. Why?
- This strategy is sometimes known as "Hibbard deletion".

Hard Challenge (Hopefully Now Easy)

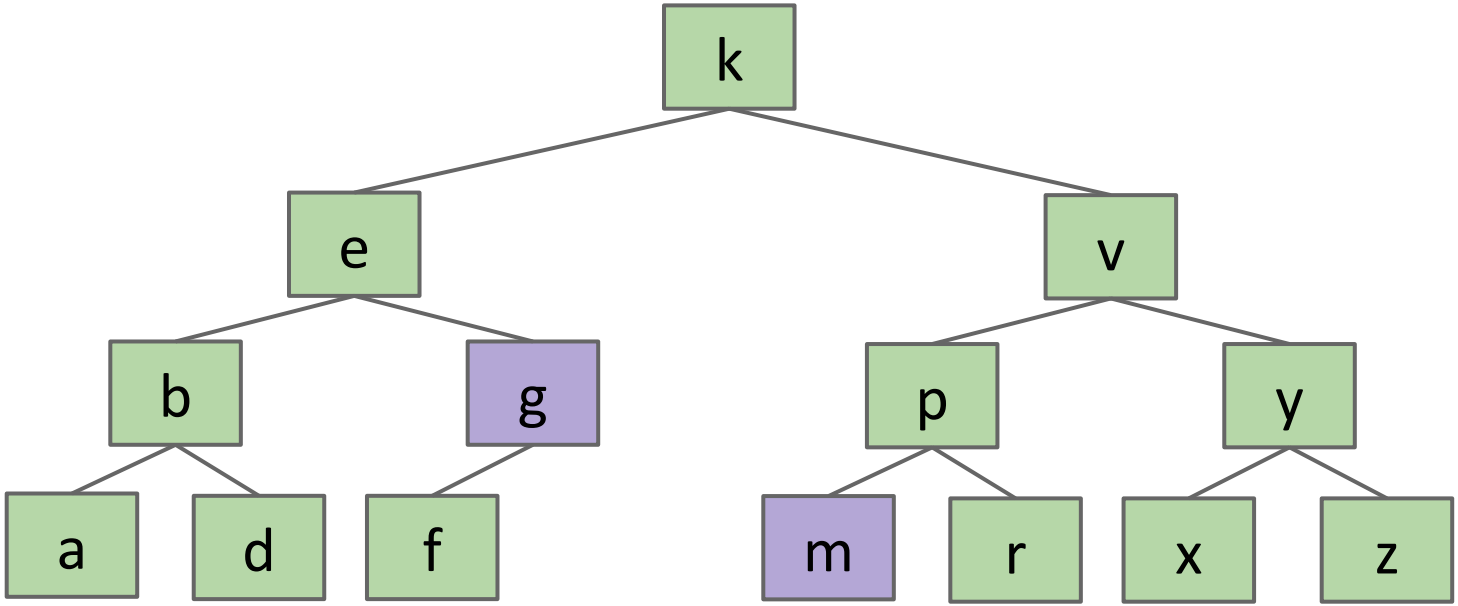
Delete k.



Hard Challenge (Hopefully Now Easy)

Delete k. Two solutions: Either promote g or m to be in the root.

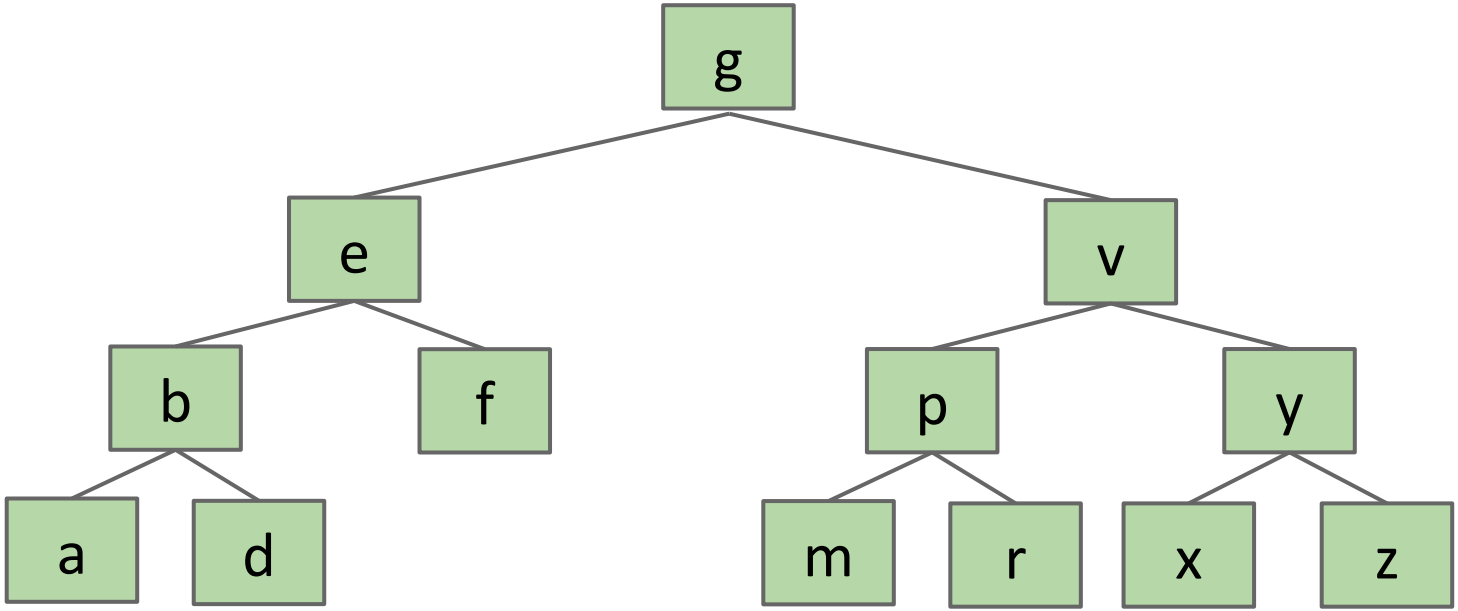
- Below, solution for g is shown.



Hard Challenge (Hopefully Now Easy)

Two solutions: Either promote g or m to be in the root.

- Below, solution for g is shown.



Sets and Maps (are the same thing)

Lecture 16, CS61B, Fall 2025

Extends

- RotatingLL
- TimeSeries

Binary Search Trees

- Today's goal: Implement the Set (and Map) ADT
- Derivation
- Definition
- contains
- Insert
- Hibbard deletion

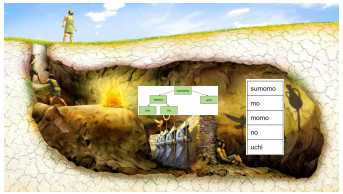
Sets and Maps (are the same thing)

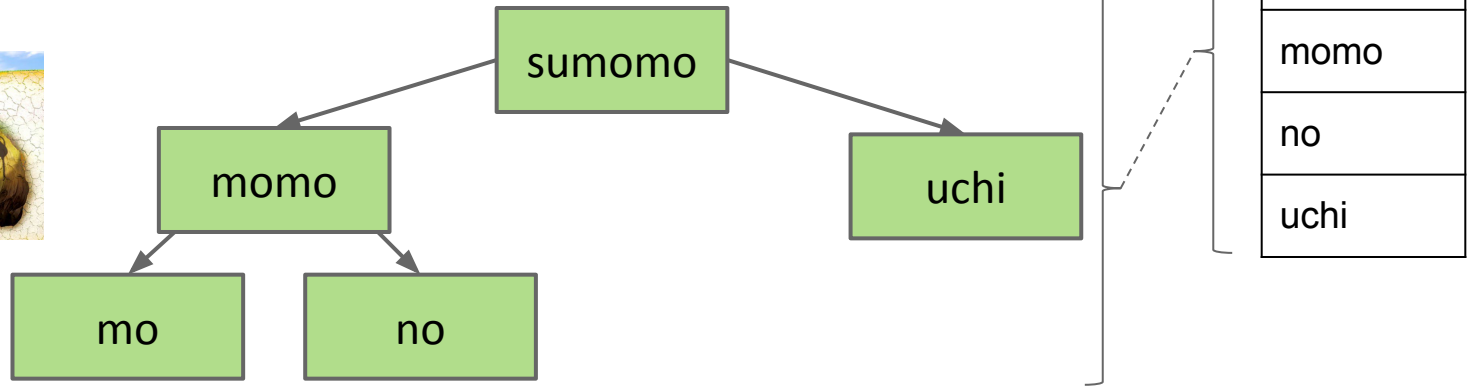
BST Implementation Tips

Sets vs. Maps

Can think of the BST below as representing a Set:

- {mo, no, sumomo, uchi, momo}

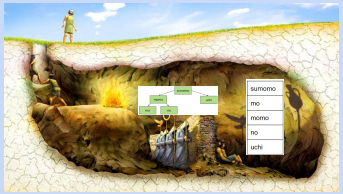


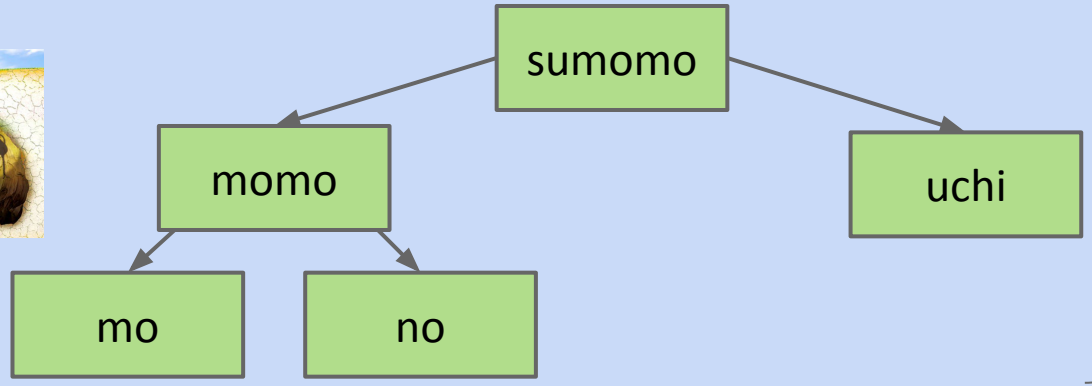


Sets vs. Maps

Can think of the BST below as representing a Set:

- {mo, no, sumomo, uchi, momo}





sumomo
mo
momo
no
uchi

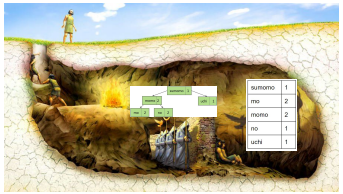
But what if we wanted to represent a mapping of word counts?

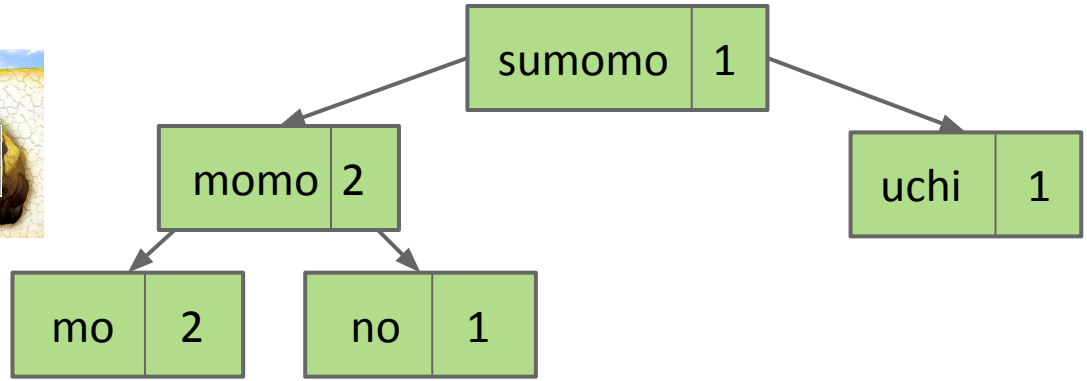
????

sumomo	1
mo	2
momo	2
no	1
uchi	1

Sets vs. Maps

To represent maps, just have each BST node store key/value pairs.





sumomo	1
mo	2
momo	2
no	1
uchi	1

Note: No efficient way to look up by value.

- Example: Cannot find all the keys with value = 1 without iterating over ALL nodes. This is fine.

Summary

Abstract data types (ADTs) are defined in terms of operations, not implementation.

Several useful ADTs: Disjoint Sets, Map, Set, List.

- Java provides Map, Set, List interfaces, along with several implementations.

We've seen two ways to implement a Set (or Map): ArraySet and using a BST.

- ArraySet: Θ(N) operations in the worst case.
- BST: Θ(log N) operations in the worst case if tree is balanced.

BST Implementations:

- Search and insert are straightforward (but insert is a little tricky).
- Deletion is more challenging. Typical approach is "Hibbard deletion".

BST Implementation Tips

Lecture 16, CS61B, Fall 2025

Extends

- RotatingLL
- TimeSeries

Binary Search Trees

- Today's goal: Implement the Set (and Map) ADT
- Derivation
- Definition
- contains
- Insert
- Hibbard deletion

Sets and Maps (are the same thing)

BST Implementation Tips

Tips for BST HW

- Code from class was “naked recursion”. Your BSTMap will not be.
- For each public method, e.g. `put(K key, V value)`, create a private recursive method, e.g. `put(K key, V value, Node n)`
- When inserting, always set left/right pointers, even if nothing is actually changing.
- Avoid “arms length base cases”. Don’t check if left or right is null!

```
static BST insert(BST T, Key ik) {
   if (T == null)
       return new BST(ik);
   if (ik < T.key)
     T.left = insert(T.left, ik);
 else if (ik > T.key)
     T.right = insert(T.right, ik);
   return T;
}
```

Always set, even if nothing changes!

Avoid “arms length base cases”.

```
if (T.left == null)
  T.left = new BST(ik);
else if (T.right == null)
  T.right = new BST(ik);
```

Attendance: www.yellkey.com/cover

HW8 due today (asymptotic analysis).

Mini-project 3 (Percolation) due Monday.

- This is the last mini-project.

HW9 (BSTs) due next Friday (Oct 9).

- Covers up through today's lecture.

Project 4A (Ngrams) due Oct 13.

- Also covers up through today's lecture.

www.yellkey.com/